

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

ОТЧЕТ

по дисциплине «Сетевое программное обеспечение»

Выполнил:
студент группы ИА-331
Любимов Кирилл Алексеевич

Преподаватель:
А.В. Лошкарев

Москва 2026

Содержание

1	Цель работы	2
2	Описание системы	2
2.1	Назначение	2
2.2	Архитектура системы	2
3	Реализация микросервисов	2
3.1	Tickets Service	2
3.1.1	Модель данных	3
3.1.2	REST API endpoints	3
3.1.3	Пример создания тикета	3
3.2	Users Service	4
3.2.1	Модель пользователя	4
3.2.2	API endpoints	4
3.2.3	Регистрация пользователя	4
3.3	Notification Service (gRPC)	5
3.3.1	Реализация gRPC сервиса	5
4	Контейнеризация с Docker	6
4.1	Dockerfile для микросервисов	6
4.2	Docker Compose	7
4.2.1	Основные сервисы	7
4.2.2	Инфраструктурные компоненты	7
4.3	API Gateway (Nginx)	8
4.4	Сети и Volumes	9
5	Запуск и тестирование системы	9
5.1	Запуск системы	9
5.2	Проверка статуса	9
5.3	Тестирование API	10
6	Выводы	10

1 Цель работы

Целью данной лабораторной работы является разработка микросервисной системы управления тикетами с использованием REST API и gRPC протоколов, а также контейнеризации приложений с помощью Docker. Работа направлена на изучение принципов построения распределенных систем и взаимодействия между микросервисами.

2 Описание системы

2.1 Назначение

Разработанная система представляет собой платформу для управления тикетами (заявками), которая позволяет:

- Создавать, просматривать, обновлять и удалять тикеты
- Управлять пользователями системы
- Назначать тикеты исполнителям
- Отслеживать статус тикетов (новый, в работе, решен, закрыт)
- Получать уведомления об изменениях через gRPC

2.2 Архитектура системы

Система построена на основе микросервисной архитектуры и состоит из трех основных сервисов:

1. **Tickets Service** — REST API для управления тикетами
2. **Users Service** — REST API для управления пользователями
3. **Notification Service** — gRPC сервис для отправки уведомлений

Дополнительные компоненты инфраструктуры:

- **PostgreSQL** — реляционная база данных для хранения данных тикетов и пользователей
- **Redis** — кэш и очередь сообщений для уведомлений
- **Nginx** — API Gateway для маршрутизации запросов

Все сервисы изолированы друг от друга и взаимодействуют через сетевые протоколы (REST/gRPC).

3 Реализация микросервисов

3.1 Tickets Service

Tickets Service является основным компонентом системы и отвечает за управление жизненным циклом тикетов. Сервис реализован на Python с использованием фреймворка FastAPI и ORM SQLAlchemy.

3.1.1 Модель данных

Основная модель данных тикета включает следующие поля:

```

1 class Ticket(Base):
2     __tablename__ = "tickets"
3
4     id = Column(Integer, primary_key=True, index=True)
5     title = Column(String(255), nullable=False)
6     description = Column(Text)
7     status = Column(String(50), nullable=False, default="new")
8     priority = Column(Integer, default=3)
9     assignee_id = Column(Integer)
10    reporter_id = Column(Integer, nullable=False)
11    created_at = Column(DateTime, default=datetime.utcnow)
12    updated_at = Column(DateTime, default=datetime.utcnow)

```

Листинг 1: Модель Ticket

Модель поддерживает четыре статуса тикета: `new`, `in_progress`, `resolved`, `closed`. Приоритет задается числом от 1 до 5.

3.1.2 REST API endpoints

Сервис предоставляет следующие endpoints:

- POST `/api/tickets` — создание нового тикета
- GET `/api/tickets` — получение списка тикетов с фильтрацией
- GET `/api/tickets/{id}` — получение тикета по ID
- PUT `/api/tickets/{id}` — обновление тикета
- PATCH `/api/tickets/{id}/status` — изменение статуса
- DELETE `/api/tickets/{id}` — удаление тикета

3.1.3 Пример создания тикета

```

1 @app.post("/api/tickets", response_model=TicketResponse)
2 async def create_ticket(
3     ticket_data: TicketCreate,
4     db: Session = Depends(get_db)
5 ):
6     #
7     if ticket_data.status not in ["new", "in_progress",
8                                   "resolved", "closed"]:
9         raise HTTPException(status_code=400,
10                               detail="Invalid status")
11
12     #
13     if ticket_data.priority < 1 or ticket_data.priority > 5:
14         raise HTTPException(status_code=400,
15                               detail="Priority must be between 1 and 5")
16
17     #
18     new_ticket = Ticket(
19         title=ticket_data.title,
20         description=ticket_data.description,

```

```

21         status=ticket_data.status,
22         priority=ticket_data.priority,
23         assignee_id=ticket_data.assignee_id,
24         reporter_id=ticket_data.reporter_id
25     )
26
27     db.add(new_ticket)
28     db.commit()
29     db.refresh(new_ticket)
30
31     #
32     if ticket_data.assignee_id:
33         asyncio.create_task(send_notification_async(
34             ticket_data.assignee_id,
35             "New Ticket Assigned",
36             f"Ticket #{new_ticket.id}: {new_ticket.title}"
37         ))
38
39     return TicketResponse.from_orm(new_ticket)

```

Листинг 2: Endpoint создания тикета

3.2 Users Service

Users Service отвечает за управление пользователями системы. Сервис предоставляет простой REST API для регистрации и получения информации о пользователях.

3.2.1 Модель пользователя

```

1 class User(Base):
2     __tablename__ = "users"
3
4     id = Column(Integer, primary_key=True, index=True)
5     username = Column(String(100), unique=True, nullable=False)
6     email = Column(String(255), unique=True, nullable=False)
7     role = Column(String(50), default="user")
8     created_at = Column(DateTime, default=datetime.utcnow)

```

Листинг 3: Модель User

Модель поддерживает три роли: admin, agent, user.

3.2.2 API endpoints

- POST /api/users/register — регистрация нового пользователя
- GET /api/users/{id} — получение пользователя по ID
- GET /api/users — получение списка пользователей

3.2.3 Регистрация пользователя

```

1 @app.post("/api/users/register", response_model=UserResponse)
2 def register(user_data: UserCreate, db: Session = Depends(get_db)):
3     #                                     username      email
4     existing_user = db.query(User).filter(
5         (User.username == user_data.username) |

```

```

6         (User.email == user_data.email)
7     ).first()
8
9     if existing_user:
10         raise HTTPException(
11             status_code=400,
12             detail="Username or email already registered"
13         )
14
15     #
16     if user_data.role not in ["admin", "agent", "user"]:
17         raise HTTPException(
18             status_code=400,
19             detail="Invalid role"
20         )
21
22     #
23     new_user = User(
24         username=user_data.username,
25         email=user_data.email,
26         role=user_data.role
27     )
28
29     db.add(new_user)
30     db.commit()
31     db.refresh(new_user)
32
33     return UserResponse.from_orm(new_user)

```

Листинг 4: Endpoint регистрации

3.3 Notification Service (gRPC)

Notification Service реализован с использованием gRPC для обеспечения высокой производительности межсервисного взаимодействия. Сервис использует Protocol Buffers для сериализации данных.

3.3.1 Реализация gRPC сервиса

```

1 class NotificationServiceServicer(
2     notifications_pb2_grpc.NotificationServiceServicer
3 ):
4     def SendNotification(self, request, context):
5         try:
6             notification_id = str(uuid.uuid4())
7
8             notification_data = {
9                 "id": notification_id,
10                "user_id": request.user_id,
11                "type": request.type,
12                "title": request.title,
13                "message": request.message,
14                "created_at": datetime.utcnow().isoformat(),
15                "read": False
16            }
17
18            # Redis
19            redis_key = f"notifications:user:{request.user_id}"

```

```

20         redis_client.lpush(redis_key,
21                             json.dumps(notification_data))
22         redis_client.ltrim(redis_key, 0, 99)
23
24         logger.info(f"Notification sent to user "
25                     f"{request.user_id}: {request.title}")
26
27         return notifications_pb2.NotificationResponse(
28             success=True,
29             notification_id=notification_id,
30             message="Notification sent successfully"
31         )
32     except Exception as e:
33         logger.error(f"Error sending notification: {e}")
34         return notifications_pb2.NotificationResponse(
35             success=False,
36             notification_id="",
37             message=f"Error: {str(e)}"
38         )

```

Листинг 5: gRPC Notification Service

Сервис сохраняет уведомления в Redis, что позволяет быстро получать историю уведомлений для каждого пользователя.

4 Контейнеризация с Docker

4.1 Dockerfile для микросервисов

Каждый микросервис упакован в отдельный Docker-образ. Пример Dockerfile для Tickets Service:

```

1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 #
6 RUN apt-get update && apt-get install -y \
7     gcc \
8     postgresql-client \
9     curl \
10    && rm -rf /var/lib/apt/lists/*
11
12 # Python
13 COPY requirements.txt .
14 RUN pip install --no-cache-dir -r requirements.txt
15
16 #
17 COPY . .
18
19 EXPOSE 8124
20
21 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8124"]

```

Листинг 6: Dockerfile для Tickets Service

Использование многоэтапной сборки и удаление кэша пакетов позволяет минимизировать размер образа.

4.2 Docker Compose

Для оркестрации всех сервисов используется Docker Compose. Конфигурация определяет все сервисы, их зависимости, сети и volumes.

4.2.1 Основные сервисы

```

1 services:
2   tickets-service:
3     build:
4       context: ./services/tickets-service
5       dockerfile: Dockerfile
6     container_name: tickets-svc-s15
7     ports:
8       - "8124:8124"
9     environment:
10      - DATABASE_URL=postgresql://tickets_user:tickets_pass@postgres:5432/
11        tickets_db
12      - REDIS_URL=redis://redis:6379/0
13      - USERS_SERVICE_URL=http://users-service:8125
14     depends_on:
15      - postgres
16      - redis
17      - users-service
18     healthcheck:
19      test: ["CMD", "curl", "-f", "http://localhost:8124/health"]
20      interval: 30s
21      timeout: 10s
22      retries: 3
23     networks:
24      - tickets-network
25
26   users-service:
27     build:
28       context: ./services/users-service
29       dockerfile: Dockerfile
30     container_name: users-svc-s15
31     ports:
32       - "8125:8125"
33     environment:
34      - DATABASE_URL=postgresql://tickets_user:tickets_pass@postgres:5432/
35        tickets_db
36     depends_on:
37      - postgres
38     networks:
39      - tickets-network

```

Листинг 7: Docker Compose - сервисы

4.2.2 Инфраструктурные компоненты

```

1 postgres:
2   image: postgres:15-alpine
3   container_name: postgres-tickets
4   environment:
5     - POSTGRES_DB=tickets_db
6     - POSTGRES_USER=tickets_user
7     - POSTGRES_PASSWORD=tickets_pass

```

```

8     volumes:
9         - postgres-data:/var/lib/postgresql/data
10        - ./init-scripts/init.sql:/docker-entrypoint-initdb.d/init.sql
11    healthcheck:
12        test: ["CMD-SHELL", "pg_isready -U tickets_user -d tickets_db"]
13        interval: 10s
14        timeout: 5s
15        retries: 5
16    networks:
17        - tickets-network
18
19    redis:
20        image: redis:7-alpine
21        container_name: redis-tickets
22        volumes:
23            - redis-data:/data
24        command: redis-server --appendonly yes
25        healthcheck:
26            test: ["CMD", "redis-cli", "ping"]
27            interval: 10s
28            timeout: 5s
29            retries: 5
30        networks:
31            - tickets-network

```

Листинг 8: PostgreSQL и Redis

Использование health checks позволяет Docker Compose корректно определять готовность сервисов к работе.

4.3 API Gateway (Nginx)

Nginx выступает в роли API Gateway, обеспечивая единую точку входа для всех клиентских запросов и маршрутизацию к соответствующим микросервисам.

```

1 http {
2     upstream tickets_backend {
3         server tickets-service:8124;
4     }
5
6     upstream users_backend {
7         server users-service:8125;
8     }
9
10    server {
11        listen 80;
12        server_name localhost;
13
14        # Tickets API
15        location /api/tickets {
16            proxy_pass http://tickets_backend;
17            proxy_set_header Host $host;
18            proxy_set_header X-Real-IP $remote_addr;
19            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
20        }
21
22        # Users API
23        location /api/users {
24            proxy_pass http://users_backend;
25            proxy_set_header Host $host;

```

```

26         proxy_set_header X-Real-IP $remote_addr;
27     }
28 }
29 }

```

Листинг 9: Конфигурация Nginx

API Gateway позволяет клиентам обращаться ко всем сервисам через единый порт 80, скрывая внутреннюю архитектуру системы.

4.4 Сети и Volumes

Docker Compose создает изолированную сеть для всех сервисов и persistent volumes для хранения данных:

```

1 volumes:
2   postgres-data:
3     driver: local
4   redis-data:
5     driver: local
6
7 networks:
8   tickets-network:
9     driver: bridge

```

Листинг 10: Сети и Volumes

Использование именованных volumes обеспечивает сохранность данных при перезапуске контейнеров.

5 Запуск и тестирование системы

5.1 Запуск системы

Для запуска всей системы достаточно одной команды:

```

1 docker compose up -d

```

Docker Compose автоматически:

- Собирает Docker-образы для всех сервисов
- Создает сеть и volumes
- Запускает контейнеры в правильном порядке (с учетом `depends_on`)
- Инициализирует базу данных
- Настраивает health checks

5.2 Проверка статуса

```

1 #
2 docker compose ps
3
4 #
5 docker compose logs -f tickets-service

```

5.3 Тестирование API

Система предоставляет Swagger UI для интерактивного тестирования API:

- Tickets Service: <http://localhost:8124/docs>
- Users Service: <http://localhost:8125/docs>

Пример создания пользователя через curl:

```
1 curl -X POST http://localhost/api/users/register \  
2 -H "Content-Type: application/json" \  
3 -d '{  
4   "username": "testuser",  
5   "email": "test@example.com",  
6   "role": "user"  
7 }'
```

Пример создания тикета:

```
1 curl -X POST http://localhost/api/tickets \  
2 -H "Content-Type: application/json" \  
3 -d '{  
4   "title": "Bug in login",  
5   "description": "Users cannot login",  
6   "status": "new",  
7   "priority": 5,  
8   "reporter_id": 1  
9 }'
```

6 Выводы

В ходе выполнения лабораторной работы была разработана микросервисная система управления тикетами. Основные достижения:

1. Спроектирована и реализована микросервисная архитектура с тремя независимыми сервисами
2. Реализованы REST API endpoints с использованием FastAPI для Tickets и Users сервисов
3. Внедрен gRPC для высокопроизводительного межсервисного взаимодействия в Notification Service
4. Все сервисы контейнеризированы с помощью Docker
5. Создана конфигурация Docker Compose для простого развертывания всей системы
6. Настроен API Gateway на базе Nginx для маршрутизации запросов
7. Реализованы health checks для мониторинга состояния сервисов
8. Использована единая база данных PostgreSQL для упрощения архитектуры

Разработанная система демонстрирует современные подходы к построению распределенных приложений. Использование Docker и Docker Compose значительно упрощает развертывание системы и обеспечивает консистентность окружения между разработкой и production.

Микросервисная архитектура позволяет независимо разрабатывать, тестировать и масштабировать отдельные компоненты системы. Использование различных протоколов взаимодействия (REST для внешнего API и gRPC для внутренней коммуникации) демонстрирует понимание их преимуществ и областей применения.

Система спроектирована с учетом принципов разделения ответственности: каждый сервис выполняет свою четко определенную функцию, что упрощает поддержку и развитие проекта.